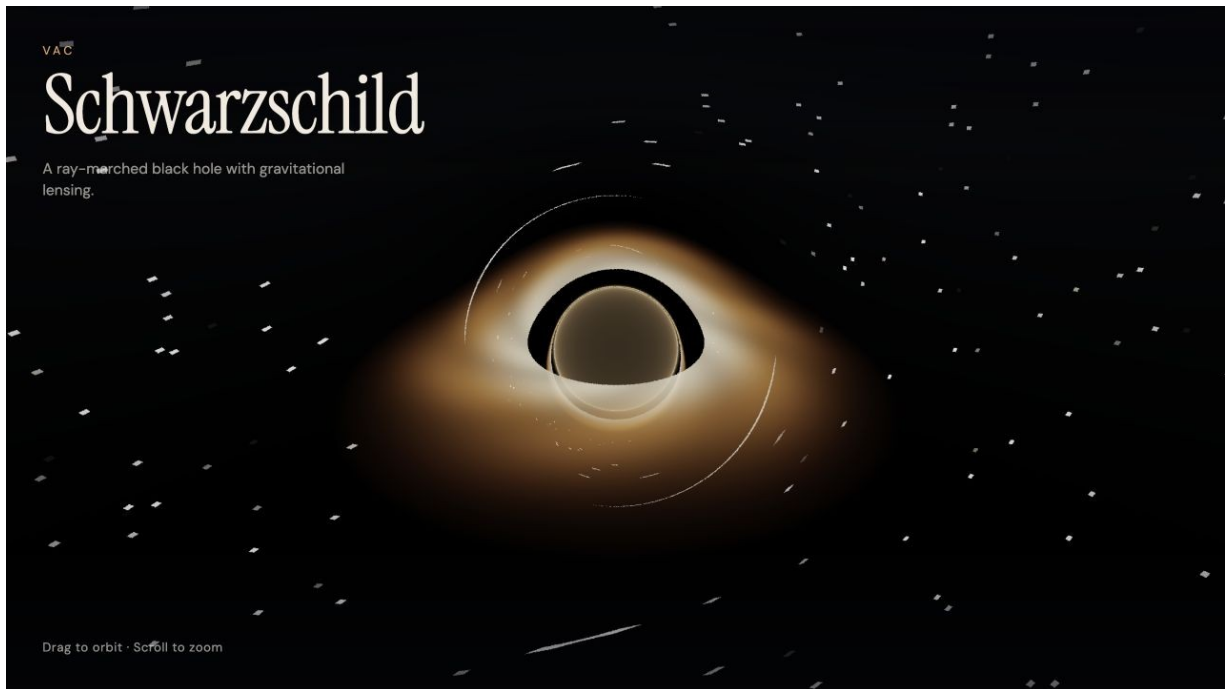


VAC TECHNICAL REPORT

Inside the VAC Schwarzschild Black Hole Simulation

Physics, mathematics, numerical integration, and the WebGL rendering pipeline



Live WebGL2 render from the current VAC code snapshot

31 July 2026

Repository branch: optimize | Commit: 6904ea0

00 Executive summary

What the project is, what it computes, and how to read the visuals

Abstract

The VAC project is a real-time WebGL2 visualization of light bending around a nonspinning Schwarzschild black hole. For every screen pixel, the fragment shader launches a camera ray, advances its spatial path through a curvature-derived central acceleration, detects crossings of a thin equatorial accretion disk, and stops when the ray is captured or escapes. The result is an interactive image with a dark capture region, lensed views of the far side of the disk, and a procedural background.

The simulation is best described as a reduced null-geodesic ray tracer on a fixed Schwarzschild spacetime. It does not evolve Einstein's field equations, solve for a changing metric, simulate disk fluid dynamics, or perform physical radiative transfer. Several visible effects - the disk spiral, photon-sphere glow, redshift falloff, starfield, tone mapping, and vignette - are intentionally artistic or heuristic.

Central takeaway: The mathematical core is compact and meaningful: the shader reproduces the Schwarzschild spatial light-orbit equation, then surrounds it with an efficient GPU rendering model and a deliberately stylized emission model.

Implementation fact	Current value	Evidence
Spacetime model	Schwarzschild, fixed $M = 1$	fragment.glsl 14-19
Ray integrator	Explicit midpoint RK2	fragment.glsl 64-71
Maximum work	260 steps per pixel	fragment.glsl 20, 105
Disk	Thin plane, $6M < r < 18M$	fragment.glsl 18-19, 129-148
Interaction	Orbit, inertia, zoom	main.js 91-155
Runtime	Six-file static WebGL2 app	Local runtime audit

Use this language

- Accurate: real-time Schwarzschild null-ray visualization; reduced spatial geodesic integration; GPU ray tracing.
- Avoid: numerical-relativity merger simulation; full 4D covariant ray tracer; physically calibrated accretion-disk model.

01 From browser input to one image

The six files divide cleanly into interface, orchestration, and GPU physics

A deliberately small architecture

1. index.html creates a full-window canvas and a minimal text overlay.
2. style.css establishes the dark visual stage, responsive title treatment, and pointer affordances.
3. main.js loads and compiles the shaders, maps user input into camera uniforms, and draws one full-screen triangle every animation frame.
4. vertex.glsl emits that triangle without a vertex buffer; each fragment receives screen coordinates.
5. fragment.glsl constructs a camera ray, integrates it, accumulates disk emission, and tone-maps the result.

The frame loop

input -> camera state -> uniforms -> full-screen draw -> one fragment program per pixel -> display

The JavaScript layer does not move geometry or evaluate the black-hole equations. It updates the camera and time, then transfers six scalar/vector uniforms to the GPU. The fragment shader owns the expensive work, which is why the same compact code can evaluate millions of rays in parallel.

Layer	Responsibility	Primary source
Presentation	Canvas, brand copy, error surface	index.html; style.css
Application	WebGL2 setup, input, camera, render loop	js/main.js
Geometry	Full-screen triangle and UV coordinates	shaders/vertex.glsl
Physics + image	Ray ODE, disk hits, background, tone map	shaders/fragment.glsl

Why one triangle: A single oversized triangle covers the viewport without a diagonal seam and avoids duplicated vertices. The actual scene exists procedurally inside the fragment shader; there are no 3D meshes or texture assets.

02 Schwarzschild geometry sets the scale

Geometric units make every distance a multiple of the black-hole mass

The background spacetime

For a spherical, nonrotating mass, the exterior geometry is described by the Schwarzschild metric. In geometric units $G = c = 1$, mass, distance, and time share compatible units:

$$ds^2 = -(1 - 2M/r) dt^2 + (1 - 2M/r)^{-1} dr^2 + r^2(d\theta^2 + \sin^2\theta d\phi^2)$$

Schwarzschild line element in standard coordinates

The shader chooses $M = 1$. To attach a physical mass later, one code length unit M corresponds to GM_{phys}/c^2 , and one code time unit corresponds to GM_{phys}/c^3 . Because the current image is scale-free, the same dimensionless geometry can represent black holes of very different masses.

Four radii organize the scene

Radius	Role	How the code uses it
$r = 2M$	Schwarzschild horizon	Capture threshold at $1.02 \times 2M$
$r = 3M$	Unstable circular photon orbit	Center of an artistic near-photon glow
$r = 6M$	ISCO for massive particles	Inner edge of the thin disk
$r = 18M$	Chosen outer disk edge	Ends the visible disk; a design parameter
$r = 70M$	Numerical escape boundary	Stops rays and samples the starfield

Important distinction: The event horizon is a spacetime boundary at $r = 2M$. The apparent shadow in an image is larger and is controlled by which light paths are captured. The bronze center visible in this version is an added glow, not a rendering of the horizon surface.

03 The light path reduces to one nonlinear orbit equation

Spherical symmetry turns a 4D geometry problem into a planar spatial curve

From the metric to the orbital equation

A photon follows a null geodesic, so $ds^2 = 0$. Spherical symmetry guarantees that each orbit lies in a plane through the origin. Choosing that plane as the equatorial plane and using the conserved energy E and angular momentum L gives a first integral for $u = 1/r$:

$$(du/d\varphi)^2 = E^2/L^2 - u^2 + 2Mu^3$$

Differentiating with respect to the orbital angle φ gives the Schwarzschild null-orbit equation:

$$d^2u/d\varphi^2 + u = 3Mu^2$$

The $3Mu^2$ term produces relativistic bending and the photon sphere

The shader's 3D vector form

The shader represents the same unparameterized spatial curve in Cartesian coordinates. With $\mathbf{r}$ -vector $\mathbf{r}$, velocity $\mathbf{v} = d\mathbf{r}/d\lambda$, and specific angular momentum $\mathbf{L} = \mathbf{r} \times \mathbf{v}$, a central acceleration that reproduces the orbit equation is:

$$d^2\mathbf{r}/d\lambda^2 = -3M |\mathbf{L}|^2 \mathbf{r} / |\mathbf{r}|^5$$

Because $RS = 2M$, the implementation coefficient $-1.5 RS$ is exactly $-3M$. The acceleration is purely radial, so the continuum trajectory remains planar and conserves the direction of $\mathbf{L}$.

```
vec3 L = cross(pos, vel);
float L2 = max(dot(L, L), 1e-8);
return -1.5 * RS * L2 * pos / (r2 * r2 * r);
```

What is exact here: For the spatial orbit shape, the central-force form is equivalent to the standard Schwarzschild null-orbit equation.

What is omitted: The shader does not integrate the Schwarzschild coordinate time, a local observer tetrad, a full four-momentum, or frequency transport. Its camera ray is initialized in a Euclidean basis.

04 Explicit midpoint RK2 advances each ray

A second-order step balances stability, cost, and WebGL simplicity

State and update

The ray state is $y = (r, v)$, with derivative $f(y) = (v, a(r, v))$. The shader uses the explicit midpoint Runge-Kutta method:

$$k_1 = f(y_n), \quad y_{\text{mid}} = y_n + (h/2)k_1, \quad y_{n+1} = y_n + h f(y_{\text{mid}})$$

Written in the code's position-velocity split, it evaluates acceleration at the current state, estimates a midpoint, evaluates acceleration again, then updates position and velocity with midpoint values. The method has second-order global accuracy when the solution is smooth and the step sequence is refined consistently.

```
a1 = geodesicAccel(pos, vel)
posMid = pos + 0.5 h vel
velMid = vel + 0.5 h a1
a2 = geodesicAccel(posMid, velMid)
pos = pos + h velMid; vel = vel + h a2
```

Curvature-aware step size

$$h(r) = \text{clamp}(0.045r, 0.025, 0.4)$$

Far from the black hole, the ray advances in larger steps. Near the strong-curvature region, the step becomes smaller. This is an efficient heuristic rather than an error-controlled adaptive integrator: it does not estimate local truncation error or reject a step.

Choice	Benefit	Tradeoff
RK2	Two acceleration evaluations; compact GLSL	Less accurate than RK4 at equal step size
h proportional to r	More samples where curvature is stronger	No explicit error tolerance
260-step cap	Predictable shader cost	Some tightly wound rays may stop before escape
Linear plane hit	Cheap disk intersection	Hit location inherits step-size error

05 Each ray ends in capture, escape, or disk emission

The tracer converts a continuous curve into a small set of rendering events

Termination logic

Condition	Interpretation	Image action
$r < 1.02 \text{ RS}$	Captured near the horizon	Set transmittance to zero and stop
$r > 70\text{M}$	Escaped the local scene	Sample the procedural starfield and stop
Disk plane crossing	Ray intersects $y = 0$	Interpolate hit, add disk emission, continue
Transmittance < 0.02	Prior hits dominate	Stop accumulating additional layers
260 steps	Work budget exhausted	Return color accumulated so far

Thin-disk intersection

A sign change between the previous y coordinate and the next y coordinate signals a crossing of the equatorial plane. Linear interpolation estimates the fractional position of the hit:

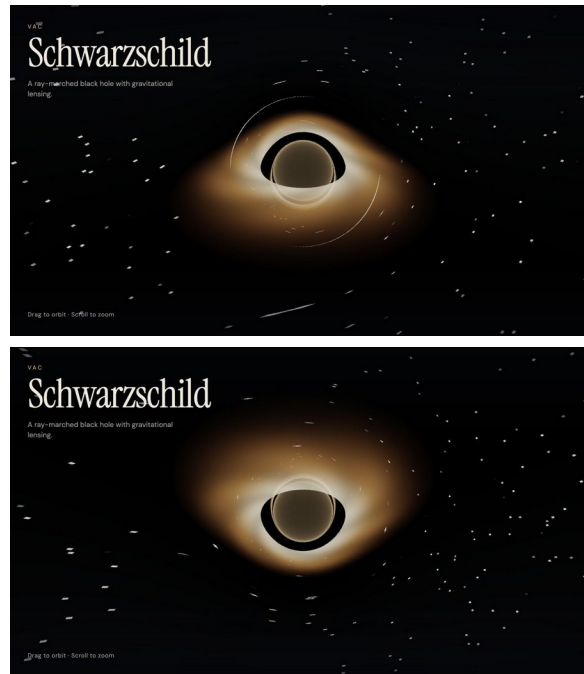
$$t_{\text{hit}} = y_{\text{prev}} / (y_{\text{prev}} - y_{\text{next}}), \quad r_{\text{hit}} = \text{mix}(r_{\text{prev}}, r_{\text{next}}, t_{\text{hit}})$$

If the cylindrical hit radius lies between 6M and 18M , the shader adds disk color and reduces transmittance to 35 percent. Continuing the ray after a hit lets one pixel collect more than one disk image when strong bending makes the path cross the plane repeatedly. That is the mechanism behind the wrapped far-side view.

Lensing is geometry, not a screen warp: The disk is not distorted after rendering. The code follows a curved ray first; the same flat disk is sampled wherever that ray crosses its plane.

06 Changing the camera reveals the lensing structure

The same scene produces different disk images because the launch direction changes



Figures 1-2. Default and near-edge-on views. Curved rays reveal direct and higher-order disk images.

Both images come from identical physics and emission code. Dragging changes azimuth and elevation; scrolling changes camera distance. The shader reconstructs a new camera basis and launches a different ray through each pixel. The strongest visual proof of lensing is the duplicated, curved image of disk material that would be hidden in a straight-line renderer.

Reading the center carefully: The black annular region and the bronze-filled center are renderer outputs, not literal surfaces. The shader adds near-photon-sphere glow after the integration loop even when a ray was captured, so the center should not be labeled as the event horizon itself.

07 The emission model turns ray events into color

Physics motivates the geometry; handcrafted functions control the visual appearance

Disk palette and spiral structure

The disk color interpolates from white-gold near the inner edge to amber and copper at larger radii. A sinusoidal pattern in azimuth, log radius, and time creates animated spiral bands. Brightness decays outward and is divided by radius:

$$B(r,\varphi,t) \propto [0.72 + 0.28 \sin(6\varphi - 8 \ln r - 1.4t)] (1 - \tau)^{1.35} / (0.18r)$$

Here τ is the normalized position between the disk's inner and outer radii. This function is visually effective, but it is not a temperature solution, fluid simulation, or emissivity law derived from disk dynamics.

Compositing and post-processing

Stage	Implementation	Scientific status
Redshift factor	Linear clamp based on hit radius	Heuristic; no frequency or observer motion
Opacity	Multiply transmittance by 0.35 per hit	Artistic thin-surface compositing
Photon glow	Accumulate near $r = 3M$	Artistic emphasis
Background	Hashed directional starfield	Procedural, not an astronomical catalog
Tone map	ACES-style rational curve	Display transform, not radiative physics
Gamma + vignette	Power 1/2.2 and radial darkening	Presentation choices

A useful design principle: Keep the geodesic solver and the emission model conceptually separate. That makes it possible to improve physical fidelity without discarding the renderer or the interaction design.

08 WebGL makes the calculation interactive

The GPU repeats the same bounded program for every pixel

Per-frame data flow

camera distance + azimuth + elevation + time + resolution -> fragment shader -> RGBA pixel

JavaScript updates the camera, clamps frame delta to 0.05 seconds, resizes the canvas, uploads uniforms, and issues one triangle draw. The shader then runs independently for every covered pixel. No depth buffer is needed because the image is generated by ray integration rather than rasterized scene geometry.

Performance-oriented choices

Choice	Why it helps	Source
Device pixel ratio capped at 1.75	Limits fragment count on high-density screens	main.js 80-89
Antialiasing disabled	Avoids redundant multisample work	main.js 177-182
Fixed subpixel jitter	Softens edges without temporal sparkle	fragment.glsl 170-178
Adaptive spatial step	Spends iterations near strong curvature	fragment.glsl 124-127
Early exits	Stops captured, escaped, or opaque rays	fragment.glsl 108-117, 145-147
High-performance context hint	Requests a performance-oriented GPU path	main.js 177-182

At a 1280 x 720 viewport and device pixel ratio 1.75, the verified canvas was 2240 x 1260: more than 2.8 million fragments per frame. Each can run up to 260 RK2 steps, although many rays exit early.

Interaction

- Drag or touch changes azimuth and elevation; elevation clamps to approximately +/-1.2 radians.
- Release produces a short exponentially damped coast.
- Wheel zoom spans 12M to 55M; the default is 28M.

09 Validation should test mathematics, numerics, and pixels

A dramatic image is evidence of execution, not sufficient evidence of correctness

Recommended validation ladder

Test	Observable	Expected signature
Shader/runtime smoke	All five assets load; WebGL2 compiles	No error overlay or console errors
Planarity	Direction of $L = r \times v$	Remains fixed within numerical error
Photon sphere	Tangential ray at $r = 3M$	Approximately circular orbit; drift shrinks with h
Weak-field deflection	Ray with large impact parameter b	Deflection approaches $4M/b$
Step convergence	Reference ray at $h, h/2, h/4$	RK2 global error approaches second order
Disk intersection	Known straight/weakly bent crossing	Hit radius converges under refinement
Visual regression	Fixed camera screenshots	No unexpected silhouette, color, or UI change
Performance	Frame time by viewport and GPU	Budget stated with hardware and resolution

Current verified baseline: The documented static-server command loaded all application assets with HTTP 200 responses, WebGL2 initialized, the error overlay remained hidden, the console reported no warnings or errors, and drag and wheel controls worked. The only HTTP blemish was a missing favicon request.

There is currently no package manifest, build system, unit test, shader test, reference integrator, or recorded visual regression suite. The validation ladder above is therefore a roadmap, not a claim that those tests already pass.

10 The scientific boundary is clear and manageable

Separating faithful geometry from proxies makes the demo easier to explain and improve

Grounded in relativity	Simplified or heuristic	Not modeled
Schwarzschild spherical geometry	Euclidean camera basis	Evolving Einstein field equations
Null spatial orbit equation	Spatial path only; no t coordinate	Binary black holes or mergers
Horizon, photon sphere, ISCO radii	Capture threshold at $1.02 R_S$	Kerr spin and frame dragging
Curved-ray disk crossings	RK2 with heuristic step rule	Disk magnetohydrodynamics
Multiple lensed disk images	Handcrafted emissivity and opacity	Physical radiative transfer
Scale-free geometric units	Procedural stars and post-processing	Doppler beaming and calibrated spectra

Recommended public description

One sentence: An interactive WebGL2 visualization that integrates reduced Schwarzschild light paths around a nonspinning black hole and samples a stylized thin accretion disk.

Why this is not full numerical relativity

Numerical relativity evolves the spacetime metric and constraints on a spatial grid. This project instead supplies an analytic Schwarzschild background and computes test-light trajectories within it. It is a substantial relativistic ray tracer, but it solves a different problem.

11 Known numerical artifacts are separate from the physics

Several visible edge cases come from work limits, stepping, or shader portability

Fixed work budget

The 260-step loop and maximum step $h = 0.4$ provide at most about 104 path-length units. A ray launched inward from the maximum 55M camera distance can require more than that to turn around and reach the 70M escape sphere. When the loop ends without capture or escape, the shader returns only accumulated color and does not add the starfield, so an escaping ray can appear spuriously dark.

Exploratory audit: A read-only emulation of the exact loop on a coarse 101 x 57 ray grid found budget exhaustion for about 1.7 percent of rays at the default 28M camera distance and about 32 percent at 55M. Continued sampled rays escaped rather than being captured. These figures are audit measurements, not a repository test or a universal screen-space rate.

Other implementation-sensitive effects

Issue	Consequence	Practical fix
Glow adds a fixed amount per step	Brightness changes when step size or step count changes	Scale by h or define an optical-depth integral
smoothstep edges are reversed	GLSL behavior is undefined when $\text{edge0} \geq \text{edge1}$	Use $1 - \text{smoothstep}(0, 0.8, \text{distance})$
Single-precision near an unstable orbit	Fine photon rings and higher-order images are resolution-sensitive	Reference against a higher-precision CPU solver
Camera can move to 12M inside the 18M disk	Observer may sit within the modeled radial disk domain	Clamp outside the disk or label this exploration mode
Capture cutoff at 2.04M	Numerical guard slightly outside the formal horizon	Quantify sensitivity and reduce with safer coordinates

These issues do not erase the correct geodesic core. They define where the current implementation should be treated as a qualitative visualization and where targeted tests can improve reliability.

12 A staged roadmap can add fidelity without losing the demo

Improve the transfer physics and validation before attempting a different class of solver

Stage	Change	Why it matters
1. Validate	Offline reference integrator, invariants, step-halving, fixed-camera image tests	Turns visual confidence into measured confidence
2. Correct transfer	Observer tetrad, frequency shift, invariant intensity transport	Makes color and brightness physically interpretable
3. Improve disk	Orbital velocity, Doppler beaming, emissivity/temperature model	Adds the strongest missing observable asymmetry
4. Add Kerr	Spin parameter, frame dragging, Kerr null geodesics	Extends the spacetime while retaining ray-tracing architecture
5. Tune quality	Error-controlled stepping, optional supersampling, performance telemetry	Balances fidelity and interactivity

Separate horizon: A full numerical-relativity solver would be a new project: it would evolve the metric, constraints, and gauge fields on a spatial grid. It should not be presented as the next incremental shader feature.

Small product improvements

- Expose M-normalized camera radius and landmark radii in an optional information panel.
- Add reset, pause, quality, and physics-versus-art toggles.
- Add a scope note and gate the post-capture center glow when a black shadow is desired.
- Add a favicon and a reproducible screenshot command.

A Code-to-math map

Where each concept lives in the current source tree

Concept	Equation or algorithm	Source location
Geometric units	$M = 1; RS = 2M$	fragment.glsl 14-19
Landmark radii	3M photon sphere; 6M ISCO	fragment.glsl 17-19
Geodesic acceleration	$a = -3ML^2 r / r ^5$	fragment.glsl 54-62
RK2	Explicit midpoint update	fragment.glsl 64-71
Camera ray	Orbiting Euclidean basis + perspective	fragment.glsl 73-95
Capture/escape	$r < 1.02RS; r > 70M$	fragment.glsl 105-118
Step size	$\text{clamp}(0.045r, 0.025, 0.4)$	fragment.glsl 124-127
Disk intersection	Plane sign change + interpolation	fragment.glsl 129-149
Disk emission	Palette, spiral, radial falloff	fragment.glsl 39-52
Post-processing	Tone map, gamma, vignette	fragment.glsl 161-188
Camera controls	Pointer orbit, inertia, wheel zoom	main.js 91-155
GPU frame	Uniform upload + triangle draw	main.js 157-175
WebGL setup	Compile/link, error handling	main.js 42-77, 177-220
Full-screen triangle	gl_VertexID -> three clip vertices	vertex.glsl 4-15

Snapshot rule: These line references match commit 6904ea0 on branch optimize. Re-check them if the shader or JavaScript changes.

B Run guide and source map

How to reproduce the demo and find the implementation evidence

Run locally

```
cd "Blackhole Simulation"
python3 -m http.server 8080
# Open http://localhost:8080
```

- Requires a browser with WebGL2 support.
- Drag with mouse or touch to orbit; scroll to zoom.
- The shader files are fetched at runtime, so opening index.html directly with file:// will fail in most browsers.

Primary project sources

ID	File	Coverage
C1	Blackhole Simulation/README.md	Lines 1-29
C2	Blackhole Simulation/js/main.js	Lines 1-223
C3	Blackhole Simulation/shaders/vertex.glsl	Lines 1-16
C4	Blackhole Simulation/shaders/fragment.glsl	Lines 1-189
C5	Blackhole Simulation/index.html	Lines 1-28
C6	Blackhole Simulation/css/style.css	Lines 1-121

C Scientific references

Primary and authoritative sources for the geometry, disk physics, and numerical method

- [1] E. Belbruno and F. Pretorius, A Dynamical System's Approach to Schwarzschild Null Geodesics, 2011. Equation 12 gives the spatial vector equation used by the shader. [Open reference](#)
- [2] J. M. Bardeen, W. H. Press, and S. A. Teukolsky, Rotating Black Holes: Locally Nonrotating Frames, Energy Extraction, and Scalar Synchrotron Radiation, *Astrophysical Journal* 178 (1972) 347-370. [Open reference](#)
- [3] D. N. Page and K. S. Thorne, Disk-Accretion onto a Black Hole. Time-Averaged Structure of Accretion Disk, *Astrophysical Journal* 191 (1974) 499-506. [Open reference](#)
- [4] C. T. Cunningham, The Effects of Redshifts and Focusing on the Spectrum of an Accretion Disk around a Kerr Black Hole, *Astrophysical Journal* 202 (1975) 788-802. [Open reference](#)
- [5] Event Horizon Telescope Collaboration, Astronomers Capture First Image of a Black Hole, 2019. Official explanation of the observed shadow and event horizon distinction. [Open reference](#)
- [6] K. Schwarzschild, On the Gravitational Field of a Mass Point According to Einstein's Theory, 1916; English translation. [Open reference](#)
- [7] S. M. Carroll, Lecture Notes on General Relativity, sections on Schwarzschild geometry and geodesics. [Open reference](#)
- [8] E. Hairer, S. P. Norsett, and G. Wanner, Solving Ordinary Differential Equations I: Nonstiff Problems, second edition. [Open reference](#)

Document scope

This report explains the implementation present in the VAC repository at the stated snapshot. Scientific sources support the background geometry and numerical framing; implementation claims are grounded in the local code and verified runtime behavior. No claim is made that the visual emission model is calibrated to observational data.